

HW 1

C++ 표준 라이브러리의 `std::vector`를 직접 구현해 보는 프로그램으로, 템플릿과 동적 메모리 관리를 활용하여 가상의 동적 배열 클래스(`lkh::Vector`)를 제작했다.

실행결과

The screenshot shows a C++ IDE with the following components:

- Explorer:** A file tree on the left showing the project structure: `Robit_intelligence_cpp` > `day_2` > `hw1`. Files include `DataFrame.hpp`, `main.cpp`, `Vector.cpp`, and `Vector.hpp`.
- Editor:** The `Vector.cpp` file is open, showing the implementation of the `lkh::Vector` class. The code includes a `namespace lkh {` block with a `return this->data_[index];` statement and a comment `// 첫 원소 위치 리턴`.
- Terminal:** The terminal window shows the compilation and execution of the program. The commands and output are as follows:

```
ay_2/hw1$ g++ -o hw1 main.cpp Vector.cpp
lkh@lkh-950XDB-951XDB-950XDY:~/Documents/Robit_intelligence_cpp/d
ay_2/hw1$ ./hw1
===== lkh::Vector test =====
push_back <n>           맨 뒤에 n 추가
at <i>                  i 번째 원소 값 읽기
begin                  첫 번째 원소
end                    끝 위치 확인
empty                  비어있는지 확인
erase <i>               i 번째 원소 삭제
insert <i> <n>           i 번째 위치에 n 삽입
size                  원소 개수
operator[] <i> <n>       i 번째 원소를 n 으로 변경
operator=              다른 벡터에 복사 후 비교
clear                  전체 삭제
print                  현재 벡터 출력
exit                  종료
=====

>> push_back 1
[ 1 ] size = 1 , capacity = 1

>> push_back 2
[ 1 2 ] size = 2 , capacity = 2

>> pushback 3
unknown command

>> unknown command

>> push_back 3
[ 1 2 3 ] size = 3 , capacity = 4

>> push_back 4
[ 1 2 3 4 ] size = 4 , capacity = 4

>> at 1
v.at(1) = 2

>> begin
*v.begin() = 1

>> end
*(v.end() - 1) = 4
v.end() - v.begin() = 4
```

The screenshot shows a C++ IDE with the following components:

- Explorer:** A file tree on the left showing the project structure: `Robit_intelligence_cpp` (containing `day_1`, `day_2`, `hw1`, `hw2`, `Day1`, and `README.md`), `hw1` (containing `DataFrame.hpp`, `hw1`, `main.cpp`, `Vector.cpp`, and `Vector.hpp`), and `hw2` (containing `DataFrame.hpp`, `main.cpp`, `Vector.cpp`, and `Vector.hpp`).
- Editor:** The `Vector.cpp` file is open, showing the implementation of the `lkh` namespace. It includes a `return this->data_[index];` statement and a comment `// 첫 위수 위치 리턴`.
- Terminal:** The terminal window shows the execution of the `./hw1` program. It displays the output of various operations on a `Vector` object, including `empty`, `erase`, `insert`, `size`, `operator`, and `print`.

```
day_2 > hw2 > Vector.cpp > {} lkh
9 namespace lkh {
76     return this->data_[index];
77 }
78
79 // 첫 위수 위치 리턴

lkh@lkh-950XDB-951XDB-950XDY:~/Documents/Robit_intelligence_cpp/d
ay_2/hw1$ ./hw1
>> empty
v.empty() = 0

>> erase 2
[ 1 2 4 ] size = 3 , capacity = 4

>> insert 2 3
[ 1 2 3 4 ] size = 4 , capacity = 4

>> size
v.size() = 4

>> operator 3 0
unknown command

>> unknown command

>> unknown command

>> operator[] 3 0
[ 1 2 3 0 ] size = 4 , capacity = 4

>> operator =
unknown command

>> unknown command

>> operator=
original : [ 1 2 3 0 ] size = 4 , capacity = 4
copy      : [ -1 2 3 0 ] size = 4 , capacity = 4

>> print
[ 1 2 3 0 ] size = 4 , capacity = 4

>> clear
[ ] size = 0 , capacity = 0

>> empty
v.empty() = 1

>> exit
lkh@lkh-950XDB-951XDB-950XDY:~/Documents/Robit_intelligence_cpp/d
ay_2/hw1$
```

중요 코드 특징

기반 클래스인 `DataFrame`을 상속받아 `Vector` 클래스를 설계했으며, 템플릿(template <class T>)을 적용하여 데이터 타입에 구애받지 않고 다양한 자료형을 담을 수 있도록 구현했다.

메모리 관리 측면에서는 배열의 공간이 부족해질 때 자동으로 용량을 2배로 확장하는

grow() 함수와 push_back, pop_back, insert, erase 등의 기능을 구현하여 표준 벡터의 동작 방식을 모사했다. 또한 복사 생성자와 대입 연산자 오버로딩을 통해 깊은 복사(Deep Copy)가 이루어지도록 안전하게 작성했다.